Milestone 2: Core Module Implementation & Prototyping

Email Automation & Alerts Platform

1. Overview

This milestone builds the three core modules from the Milestone 1 architecture — email composition, SMTP connection handling, and a scheduling engine — as small, independently testable units. A prototype wires all three together to schedule and deliver a single email end to end, proving the mechanics work before they are wrapped in the full FastAPI service.

Each module was unit-tested and the prototype was actually executed. Section 5 is the real console output from that run, including a live local SMTP server receiving the message.

Module responsibilities

Module	Responsibility
email_composer.py	Builds a valid MIME message from plain inputs. No network code, so it is pure and testable.
smtp_client.py	Owns the SMTP connection: TLS/auth, retry with backoff, context-managed lifecycle.
scheduler_engine.py	Thin wrapper around APScheduler exposing schedule_once / interval / cron.
prototype_demo.py	Wires the three together against a local throwaway SMTP server.

2. email_composer.py

Builds a `ComposedEmail` from plain inputs. It validates before anything touches SMTP, and forces quoted-printable encoding so bodies stay human-readable instead of base64 blobs (a bug caught during testing: Python base64-encodes UTF-8 bodies by default).

```
"""
Email composition module.

Builds well-formed MIME email messages ready to hand to an SMTP client.
Deliberately free of any network code so it's independently testable --
composing a message never touches a socket.
"""

from __future__ import annotations

import email.charset as _email_charset
from dataclasses import dataclass
from email.mime.application import MIMEApplication
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.utils import formataddr, make_msgid
from pathlib import Path
from typing import List, Optional, Sequence

# By default, Python's email module base64-encodes UTF-8 text bodies.
# That's valid MIME but makes every message an unreadable blob -- annoying
# for logs, debugging, and anyone inspecting a raw .eml file. Quoted-printable
# keeps plain ASCII content readable as-is while still handling non-ASCII
# characters correctly.
_email_charset.add_charset("utf-8", _email_charset.QP, _email_charset.QP, "utf-8")


class EmailCompositionError(ValueError):
```

```
    """Raised when an email cannot be composed from the given inputs."""


@dataclass
class Attachment:
    filename: str
    content: bytes
    mime_subtype: str = "octet-stream"


@dataclass
class ComposedEmail:
    """A fully-built email, ready for an SMTP client to send."""

    sender: str
    recipients: List[str]
    cc: List[str]
    subject: str
    message: MIMEMultipart

    @property
    def all_recipients(self) -> List[str]:
        return [*self.recipients, *self.cc]

    def as_string(self) -> str:
        return self.message.as_string()


def compose_email(
    sender: str,
    recipients: Sequence[str],
    subject: str,
    body: str,
    *,
    sender_name: Optional[str] = None,
    cc: Optional[Sequence[str]] = None,
    is_html: bool = False,
    attachments: Optional[Sequence[Attachment]] = None,
) -> ComposedEmail:
    """Build a ComposedEmail from plain inputs.

    Raises EmailCompositionError on a missing subject/body or an empty
    recipient list, so bad input is caught before anything touches SMTP.
    """
    recipients = list(recipients or [])
    cc = list(cc or [])

    if not recipients:
        raise EmailCompositionError("At least one recipient is required")
    if not subject.strip():
        raise EmailCompositionError("Subject cannot be empty")
    if not body.strip():
        raise EmailCompositionError("Body cannot be empty")

    msg = MIMEMultipart("mixed")
    msg["From"] = formataddr((sender_name, sender)) if sender_name else sender
    msg["To"] = ", ".join(recipients)
    if cc:
        msg["Cc"] = ", ".join(cc)
    msg["Subject"] = subject
    msg["Message-ID"] = make_msgid()

    alt = MIMEMultipart("alternative")
    alt.attach(MIMEText(body, "html" if is_html else "plain", "utf-8"))
    msg.attach(alt)

    for att in attachments or []:
        part = MIMEApplication(att.content, _subtype=att.mime_subtype)
        part.add_header("Content-Disposition", "attachment", filename=att.filename)
        msg.attach(part)

    return ComposedEmail(sender=sender, recipients=recipients, cc=cc, subject=subject, message=msg)


def attachment_from_file(path: str) -> Attachment:
    """Convenience: build an Attachment by reading a file from disk."""
    p = Path(path)
```

```
    subtype = p.suffix.lstrip(".") or "octet-stream"
    return Attachment(filename=p.name, content=p.read_bytes(), mime_subtype=subtype)
```

3. smtp_client.py

A context-managed `SMTPClient`: handles STARTTLS and login, retries transient failures with backoff, and fails fast on non-transient SMTP errors.

```
"""
SMTP connection handling module.

Wraps smtplib with a small, reusable client: a context-managed connection,
TLS/auth handling, and retry-with-backoff on transient failures. Callers
never touch smtplib directly.
"""

from __future__ import annotations

import logging
import smtplib
import ssl
import time
from dataclasses import dataclass
from typing import Optional

from email_composer import ComposedEmail

logger = logging.getLogger("automation.smtp_client")


class SMTPSendError(Exception):
    """Raised after all retry attempts to send a message have failed."""


@dataclass
class SMTPConfig:
    host: str
    port: int = 587
    username: Optional[str] = None
    password: Optional[str] = None
    use_tls: bool = True
    timeout: int = 30
    max_retries: int = 3
    retry_backoff_seconds: float = 2.0


class SMTPClient:
    """A small, reusable SMTP client.

    Usage:
        with SMTPClient(config) as client:
            client.send(composed_email)
    """

    def __init__(self, config: SMTPConfig):
        self._config = config
        self._connection: Optional[smtplib.SMTP] = None

    def __enter__(self) -> "SMTPClient":
        self.connect()
        return self

    def __exit__(self, exc_type, exc, tb) -> None:
        self.close()

    def connect(self) -> None:
        cfg = self._config
        self._connection = smtplib.SMTP(cfg.host, cfg.port, timeout=cfg.timeout)
        self._connection.ehlo()
        if cfg.use_tls:
            context = ssl.create_default_context()
            self._connection.starttls(context=context)
            self._connection.ehlo()
        if cfg.username and cfg.password:
            self._connection.login(cfg.username, cfg.password)
```

```
        logger.info("Connected to SMTP server %s:%s", cfg.host, cfg.port)

    def close(self) -> None:
        if self._connection is not None:
            try:
                self._connection.quit()
            except Exception:  # noqa: BLE001 - connection may already be dead
                self._connection.close()
            finally:
                self._connection = None

    def send(self, email: ComposedEmail) -> None:
        """Send one composed email, retrying transient failures."""
        if self._connection is None:
            raise SMTPSendError("Not connected -- use `with SMTPClient(...) as client:`")

        last_error: Optional[Exception] = None
        for attempt in range(1, self._config.max_retries + 1):
            try:
                self._connection.sendmail(email.sender, email.all_recipients, email.as_string())
                logger.info("Sent '%s' to %s (attempt %d)", email.subject, email.all_recipients, attempt)
                return
            except (smtplib.SMTPServerDisconnected, smtplib.SMTPConnectError, ConnectionError) as exc:
                last_error = exc
                logger.warning("Transient SMTP error on attempt %d: %s", attempt, exc)
                if attempt < self._config.max_retries:
                    time.sleep(self._config.retry_backoff_seconds * attempt)
                    self._reconnect()
            except smtplib.SMTPException as exc:
                last_error = exc
                break  # not transient - retrying won't help

        raise SMTPSendError(f"Failed to send '{email.subject}' after retries: {last_error}") from last_error

    def _reconnect(self) -> None:
        self.close()
        self.connect()
```

4. scheduler_engine.py

Wraps APScheduler's `BackgroundScheduler` behind a small interface so callers never import APScheduler directly. Moving to Celery Beat later means rewriting only this file.

```
"""
Scheduling engine module.

A thin, swappable wrapper around APScheduler. Calling code only ever talks
to `SchedulerEngine` -- so the underlying scheduler (in-process APScheduler
today, Celery Beat later, per the Milestone 1 architecture) can change
without touching any caller.
"""

from __future__ import annotations

import logging
from datetime import datetime
from typing import Any, Callable, Optional

from apscheduler.schedulers.background import BackgroundScheduler
from apscheduler.triggers.cron import CronTrigger
from apscheduler.triggers.date import DateTrigger
from apscheduler.triggers.interval import IntervalTrigger

logger = logging.getLogger("automation.scheduler_engine")


class SchedulerEngine:
    """Core scheduling engine: register jobs, check their next run time, cancel them."""

    def __init__(self):
        self._scheduler = BackgroundScheduler()

    def start(self) -> None:
        if not self._scheduler.running:
            self._scheduler.start()
```

```
            logger.info("Scheduler engine started")

    def shutdown(self, wait: bool = False) -> None:
        if self._scheduler.running:
            self._scheduler.shutdown(wait=wait)
            logger.info("Scheduler engine stopped")

    def schedule_once(self, job_id: str, run_at: datetime, func: Callable, *args: Any, **kwargs: Any) -> str:
        """Schedule `func` to run exactly once, at `run_at`."""
        self._scheduler.add_job(
            func, trigger=DateTrigger(run_date=run_at), id=job_id,
            args=args, kwargs=kwargs, replace_existing=True, misfire_grace_time=300,
        )
        return job_id

    def schedule_interval(self, job_id: str, seconds: int, func: Callable, *args: Any, **kwargs: Any) -> str:
        """Schedule `func` to run every `seconds` seconds, forever."""
        self._scheduler.add_job(
            func, trigger=IntervalTrigger(seconds=seconds), id=job_id,
            args=args, kwargs=kwargs, replace_existing=True, misfire_grace_time=60,
        )
        return job_id

    def schedule_cron(self, job_id: str, cron_expression: str, func: Callable, *args: Any, **kwargs: Any) -> str:
        """Schedule `func` on a 5-field crontab expression."""
        self._scheduler.add_job(
            func, trigger=CronTrigger.from_crontab(cron_expression), id=job_id,
            args=args, kwargs=kwargs, replace_existing=True, misfire_grace_time=300,
        )
        return job_id

    def cancel(self, job_id: str) -> None:
        if self._scheduler.get_job(job_id):
            self._scheduler.remove_job(job_id)

    def next_run_time(self, job_id: str) -> Optional[datetime]:
        """Returns None once a one-time job has fired, or if job_id doesn't exist."""
        job = self._scheduler.get_job(job_id)
        if job and job.next_run_time:
            return job.next_run_time.replace(tzinfo=None)
        return None
```

5. Prototype: single-message scheduling, demonstrated live

The prototype starts a throwaway local SMTP server (`aiosmtpd`) that captures what it receives, so the full pipeline (compose, schedule, fire, connect, send, deliver) is proven with no real credentials.

```
"""
Milestone 2 prototype: schedule ONE email and watch it actually get sent.

Runs a local, throwaway SMTP server (no real credentials needed) so the
whole pipeline -- compose -> schedule -> fire -> send -> deliver -- is
proven end to end in a single run.
"""

import logging
import time
from datetime import datetime, timedelta

from aiosmtpd.controller import Controller

from email_composer import compose_email
from scheduler_engine import SchedulerEngine
from smtp_client import SMTPClient, SMTPConfig

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(name)s: %(message)s")
logger = logging.getLogger("prototype")

received_messages = []


class CapturingHandler:
    """A minimal aiosmtpd handler that just records what it receives."""

    async def handle_DATA(self, server, session, envelope):
```

```
        received_messages.append({
            "mail_from": envelope.mail_from,
            "rcpt_tos": envelope.rcpt_tos,
            "content": envelope.content.decode("utf-8", errors="replace"),
        })
        logger.info("Test SMTP server received a message for %s", envelope.rcpt_tos)
        return "250 Message accepted for delivery"


def send_prototype_email():
    """The job the scheduler calls when the due time arrives."""
    email = compose_email(
        sender="automation@example.com",
        sender_name="Automation Platform",
        recipients=["stakeholder@example.com"],
        subject="Milestone 2 prototype -- single scheduled email",
        body="This message was composed, scheduled, and delivered by the Milestone 2 prototype.",
    )
    config = SMTPConfig(host="127.0.0.1", port=1025, use_tls=False)
    with SMTPClient(config) as client:
        client.send(email)


def main():
    # 1. Start a local test SMTP server -- stands in for a real mail relay
    controller = Controller(CapturingHandler(), hostname="127.0.0.1", port=1025)
    controller.start()
    logger.info("Test SMTP server listening on 127.0.0.1:1025")

    # 2. Start the scheduling engine and schedule exactly ONE email, 5s out
    engine = SchedulerEngine()
    engine.start()
    run_at = datetime.now() + timedelta(seconds=5)
    engine.schedule_once("prototype-email", run_at, send_prototype_email)
    logger.info("Scheduled prototype email for %s", run_at.strftime("%H:%M:%S"))

    # 3. Wait for it to fire (next_run_time goes to None once a one-time job runs)
    waited = 0.0
    while engine.next_run_time("prototype-email") is not None and waited < 15:
        time.sleep(0.5)
        waited += 0.5
    time.sleep(0.5)  # let the job function finish sending

    # 4. Verify delivery
    if received_messages:
        msg = received_messages[0]
        logger.info("SUCCESS -- email delivered to %s", msg["rcpt_tos"])
        print("\n--- Message received by the test SMTP server ---")
        print(msg["content"])
    else:
        logger.error("FAILED -- no message was received by the test SMTP server")

    engine.shutdown()
    controller.stop()


if __name__ == "__main__":
    main()
```

Actual run output

From `python3 prototype_demo.py`. The job was scheduled at 16:59:18 for 16:59:23 (5 seconds out) and fired on time. Log lines are condensed to the key events.

```
16:59:18  Test SMTP server listening on 127.0.0.1:1025
16:59:18  Scheduler engine started
16:59:18  Scheduled prototype email for 16:59:23
          ... (5 second wait) ...
16:59:23  Running job "send_prototype_email"
16:59:23  Connected to SMTP server 127.0.0.1:1025
16:59:23  Sent 'Milestone 2 prototype -- single scheduled email' (attempt 1)
16:59:23  SUCCESS -- email delivered to ['stakeholder@example.com']

--- Message received by the test SMTP server ---
From: Automation Platform <automation@example.com>
To: stakeholder@example.com
Subject: Milestone 2 prototype -- single scheduled email
Content-Transfer-Encoding: quoted-printable

This message was composed, scheduled, and delivered by the Milestone 2 prot=
otype.
```

6. Test results

Five sanity tests cover composition validation and scheduler bookkeeping (`python3 tests.py`):

```
PASS: test_compose_basic_email
PASS: test_compose_rejects_empty_recipients
PASS: test_compose_rejects_empty_subject
PASS: test_compose_with_attachment
PASS: test_scheduler_registers_and_reports_next_run

All tests passed.
```

7. Next steps (Milestone 3)

- Wrap these modules in the FastAPI layer: validation, persistence, REST endpoints
- Replace the throwaway test SMTP server with real credentials via environment configuration
- Move the scheduler job store from in-memory to the database so schedules survive a restart
- Add the alert-checking engine alongside the email-sending engine